

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('/content/Crop Recommendation using Soil Properties and Weather Prediction (2).csv')
```

```
df.head()
```

```

Soilcolor  Ph      K      P      N      Zn      S  QV2M-W  QV2M-Sp  QV2M-Su  ...  PRECTOTCORR-W  PRECTOTCORR-Sp  PRECTOTCORR-S
0  Yellowish brown  5.81  738.231  5.401  0.23  2.976000  13.816  7.993333  10.456667  11.963333  ...  2.073333  5.27  12.303333
1  Yellowish brown  5.43  606.382  10.478  0.23  3.077000  16.421  7.993333  10.456667  11.963333  ...  2.073333  5.27  12.303333
2    brown  5.41  386.580  6.847  0.23  6.611000  16.557  7.993333  10.456667  11.963333  ...  2.073333  5.27  12.303333
3    red  5.65  207.086  3.418  0.23  0.460181  16.075  7.993333  10.456667  11.963333  ...  2.073333  5.27  12.303333
4    red  5.27  317.357  39.282  0.23  2.743000  12.558  7.993333  10.456667  11.963333  ...  2.073333  5.27  12.303333
5 rows x 29 columns

```

```
df.shape
```

```
(3867, 29)
```

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3867 entries, 0 to 3866
Data columns (total 29 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Soilcolor             3867 non-null  object
 1   Ph                    3867 non-null  float64
 2   K                     3867 non-null  float64
 3   P                     3867 non-null  float64
 4   N                     3867 non-null  float64
 5   Zn                    3867 non-null  float64
 6   S                     3867 non-null  float64
 7   QV2M-W                3867 non-null  float64
 8   QV2M-Sp               3867 non-null  float64
 9   QV2M-Su               3867 non-null  float64
10  QV2M-Au               3867 non-null  float64
11  T2M_MAX-W             3867 non-null  float64
12  T2M_MAX-Sp            3867 non-null  float64
13  T2M_MAX-Su            3867 non-null  float64
14  T2M_MAX-Au            3867 non-null  float64
15  T2M_MIN-W             3867 non-null  float64
16  T2M_MIN-Sp            3867 non-null  float64
17  T2M_MIN-Su            3867 non-null  float64
18  T2M_MIN-Au            3867 non-null  float64
19  PRECTOTCORR-W         3867 non-null  float64
20  PRECTOTCORR-Sp        3867 non-null  float64
21  PRECTOTCORR-Su        3867 non-null  float64
22  PRECTOTCORR-Au        3867 non-null  float64
23  WD10M                 3867 non-null  float64
24  GWETTOP               3867 non-null  float64
25  CLOUD_AMT             3867 non-null  float64
26  WS2M_RANGE            3867 non-null  float64
27  PS                     3867 non-null  float64
28  label                  3867 non-null  object
dtypes: float64(27), object(2)
memory usage: 876.2+ KB

```

```

#missing values
df.isna().sum()

```



	0
Soilcolor	0
Ph	0
K	0
P	0
N	0
Zn	0
S	0
QV2M-W	0
QV2M-Sp	0
QV2M-Su	0
QV2M-Au	0
T2M_MAX-W	0
T2M_MAX-Sp	0
T2M_MAX-Su	0
T2M_MAX-Au	0
T2M_MIN-W	0
T2M_MIN-Sp	0
T2M_MIN-Su	0
T2M_MIN-Au	0
PRECTOTCORR-W	0
PRECTOTCORR-Sp	0
PRECTOTCORR-Su	0
PRECTOTCORR-Au	0
WD10M	0
GWETTOP	0
CLOUD_AMT	0
WS2M_RANGE	0
PS	0
label	0

dtype: int64

```
#checking categorical value of label  
df["label"].value_counts()
```



	count
label	
Teff	1260
Maize	732
Wheat	715
Barley	503
Bean	253
Pea	94
Sorghum	72
Dagussa	71
Niger seed	64
Potato	48
Red Pepper	29
Fallow	26

dtype: int64

```
#checking categorical value of Soil Color  
df["Soilcolor"].value_counts()
```



	count
Soilcolor	
brown	1297
red	1195
black	860
other	128
gray	88
reddish brown	85
dark brown	51
dark gray	35
Reddish brown	14
dark reddish brown	13
Red	11
Dark brown	9
darkbrown	8
Black	7
Redish brown	7
very dark brown	7
Yellowish brown	6
Brown	5
Very dark brown	4
Reddis brown	3
Grayish brown	3
dark grayish brown	3
Dark gray	2
red;luvisols	2
Dark Brown	2
very dark gray	2
Reddish gray	2
Redish Brown	1
Dark greyish brown	1
Lihatish brown	1

#coverting to correct format

```
df["Soilcolor"] = df["Soilcolor"].replace('Red', 'red')
df["Soilcolor"] = df["Soilcolor"].replace('Brown', 'brown')
df["Soilcolor"] = df["Soilcolor"].replace('Black', 'black')
df["Soilcolor"] = df["Soilcolor"].replace('redish brown', 'reddish brown')
df["Soilcolor"] = df["Soilcolor"].replace('reddis brown', 'reddish brown')
df["Soilcolor"] = df["Soilcolor"].replace('reddish brown;moist', 'reddish brown')
df["Soilcolor"] = df["Soilcolor"].replace('reddish broown', 'reddish brown')
df["Soilcolor"] = df["Soilcolor"].replace('redishbrown', 'reddish brown')
df["Soilcolor"] = df["Soilcolor"].replace('darkbrown', 'dark brown')
df["Soilcolor"] = df["Soilcolor"].replace('replacement of inaccessible target red;luvisols', 'red;luvisols')
df["Soilcolor"] = df["Soilcolor"].replace('dark greyish brown', 'dark grayish brown')
```

#converting to lower case

```
df['Soilcolor'] = df['Soilcolor'].str.lower()
```

#checking categorical value of Soil Color

```
df["Soilcolor"].value_counts()
```



	count
Soilcolor	
brown	1302
red	1206
black	867
other	128
reddish brown	100
gray	88
dark brown	70
dark gray	37
dark reddish brown	13
very dark brown	11
redish brown	8
yellowish brown	7
grayish brown	5
reddis brown	3
dark grayish brown	3
red;luvisols	3
very dark gray	2
reddish gray	2
dark greyish brown	1
lihtish brown	1
dark red	1
reddish broown	1
dark grayish	1
red brown	1
reddish	1
grayish brown (gb)	1
black;vertisol	1
perl brown	1
light red	1
redishbrown	1

#Create a dictionary to correct spelling mistakes

```
spelling_corrections = {
    'lihtish brown': 'lightish brown',
    'perl brown': 'pearl brown',
    'dark greyish brown': 'dark grayish brown',
    'grayish brown (gb)': 'grayish brown',
    'reddish' : 'red',
    'dark grayish': 'dark gray',
    'black;vertisol': 'black'
}
```

#Replace misspelled entries

```
df["Soilcolor"] = df["Soilcolor"].replace(spelling_corrections)
```

#Create a mapping dictionary for replacements

```
color_mapping = {
    'Red': 'red',
    'reddish': 'reddish',
    'reddish brown': 'reddish brown',
    'dark reddish brown': 'dark reddish brown',
    'very dark brown': 'very dark brown',
    'dark brown': 'dark brown',
}
```

```

'yellowish brown': 'yellowish brown',
'grayish brown': 'grayish brown',
'dark grayish brown': 'dark grayish brown',
'dark greyish brown': 'dark grayish brown',
'gray': 'gray',
'dark gray': 'dark gray',
'very dark gray': 'very dark gray',
'reddish gray': 'reddish gray',
'light red': 'light red',
'red brown': 'reddish brown',
'lihgtish brown': 'lightish brown',
'perl brown': 'pearl brown',
'replacement of inaccessible target red;luvisols': 'red',
'red;luvisols': 'red',
'black;vertisol': 'black',
}

```

```

#Replace values in the Soilcolor column
df["Soilcolor"] = df["Soilcolor"].replace(color_mapping)

```

```

#checking categorical value of Soil Color
df["Soilcolor"].value_counts()

```



	count
Soilcolor	
brown	1302
red	1210
black	868
other	128
reddish brown	101
gray	88
dark brown	70
dark gray	38
dark reddish brown	13
very dark brown	11
redish brown	8
yellowish brown	7
grayish brown	6
dark grayish brown	4
reddis brown	3
reddish gray	2
very dark gray	2
light red	1
pearl brown	1
reddish broown	1
dark red	1
lightish brown	1
redishbrown	1

dtype: int64

```

#adding to another csv
df.to_csv('cleaned_data.csv')

```

```

#
data = pd.read_csv('/content/cleaned_data.csv')

```

```
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)
```

```
data.head()
```

	Unnamed: 0	Soilcolor	Ph	K	P	N	Zn	S	QV2M-W	QV2M-Sp	QV2M-Su	QV2M-Au	T2M_MAX-W	T2M_MAX-Sp	T2M_MAX-Su
0	0	yellowish brown	5.81	738.231	5.401	0.23	2.976000	13.816	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
1	1	yellowish brown	5.43	606.382	10.478	0.23	3.077000	16.421	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
2	2	brown	5.41	386.580	6.847	0.23	6.611000	16.557	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
3	3	red	5.65	207.086	3.418	0.23	0.460181	16.075	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
4	4	red	5.27	317.357	39.282	0.23	2.743000	12.558	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06

```
data.describe()
```

	Unnamed: 0	Ph	K	P	N	Zn	S	QV2M-W	QV2M-Sp	QV2M-Su	T2M_MAX-Su
count	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000	3867.000000
mean	1933.000000	5.857295	324.284770	11.349588	0.179153	1.774094	11.311625	8.346870	9.082508	12.232020	10.456667
std	1116.451074	0.676690	202.250133	34.141864	0.066476	1.460809	5.542094	0.613948	0.790654	0.894970	0.894970
min	0.000000	4.300000	41.134000	0.000000	0.000262	0.100000	0.050000	7.183333	7.650000	10.476667	10.476667
25%	966.500000	5.390000	191.000000	2.000000	0.131200	1.100000	7.305000	7.933333	8.403333	11.430000	10.456667
50%	1933.000000	5.780000	282.000000	4.000000	0.179884	1.500000	10.700000	8.383333	9.340000	12.166667	10.456667
75%	2899.500000	6.200000	405.000000	7.920000	0.230000	2.060000	14.195500	8.910000	9.480000	12.836667	10.456667
max	3866.000000	8.500000	2119.000000	782.000000	0.695600	45.500000	118.347000	9.723333	10.703333	13.853333	13.853333

```
# Renaming columns
```

```
data = data.rename(columns={"Unnamed: 0": "Index"})
```

```
data.head()
```

	Index	Soilcolor	Ph	K	P	N	Zn	S	QV2M-W	QV2M-Sp	QV2M-Su	QV2M-Au	T2M_MAX-W	T2M_MAX-Sp	T2M_MAX-Su
0	0	yellowish brown	5.81	738.231	5.401	0.23	2.976000	13.816	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
1	1	yellowish brown	5.43	606.382	10.478	0.23	3.077000	16.421	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
2	2	brown	5.41	386.580	6.847	0.23	6.611000	16.557	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
3	3	red	5.65	207.086	3.418	0.23	0.460181	16.075	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06
4	4	red	5.27	317.357	39.282	0.23	2.743000	12.558	7.993333	10.456667	11.963333	9.683333	26.853333	28.526667	23.06

```
data.isna().sum()
```



	0
Index	0
Soilcolor	0
Ph	0
K	0
P	0
N	0
Zn	0
S	0
QV2M-W	0
QV2M-Sp	0
QV2M-Su	0
QV2M-Au	0
T2M_MAX-W	0
T2M_MAX-Sp	0
T2M_MAX-Su	0
T2M_MAX-Au	0
T2M_MIN-W	0
T2M_MIN-Sp	0
T2M_MIN-Su	0
T2M_MIN-Au	0
PRECTOTCORR-W	0
PRECTOTCORR-Sp	0
PRECTOTCORR-Su	0
PRECTOTCORR-Au	0
WD10M	0
GWETTOP	0
CLOUD_AMT	0
WS2M_RANGE	0
PS	0
label	0

dtype: int64

data.columns



```
Index(['Index', 'Soilcolor', 'Ph', 'K', 'P', 'N', 'Zn', 'S', 'QV2M-W',
      'QV2M-Sp', 'QV2M-Su', 'QV2M-Au', 'T2M_MAX-W', 'T2M_MAX-Sp',
      'T2M_MAX-Su', 'T2M_MAX-Au', 'T2M_MIN-W', 'T2M_MIN-Sp', 'T2M_MIN-Su',
      'T2M_MIN-Au', 'PRECTOTCORR-W', 'PRECTOTCORR-Sp', 'PRECTOTCORR-Su',
      'PRECTOTCORR-Au', 'WD10M', 'GWETTOP', 'CLOUD_AMT', 'WS2M_RANGE', 'PS',
      'label'],
      dtype='object')
```

```
crops_features = data.drop(columns=['Soilcolor','label'], axis=1)
```

```
# Build statistical outlier detection function
```

```
# Separate dataset features for EDA
```

```
def detect_outliers(dataset_features):
```

```
    """
```

```
    Detect outliers in the dataset using the Interquartile Range (IQR) method.
```

```
    Args:
```

```
        dataset_features (pd.DataFrame): Input DataFrame containing numerical features to analyze
```

```
    Returns:
```

```

pd.DataFrame: DataFrame containing for each feature:
    - Lower Bound: Q1 - 1.5*IQR threshold for outliers
    - Upper Bound: Q3 + 1.5*IQR threshold for outliers
    - Outliers Count: Number of values outside the bounds

Notes:
    - Uses the standard 1.5*IQR rule: points below Q1 - 1.5*IQR or above Q3 + 1.5*IQR are considered outliers
    """"
outliers = pd.DataFrame(index=crops_features.columns)
# Calculate quantiles
Q1 = dataset_features.quantile(0.25)
Q3 = dataset_features.quantile(0.75)
# Calculate IQR range
IQR = Q3 - Q1
# Define upper and lower bounds
outliers['Lower Bound'] = Q1 - 1.5*IQR
outliers['Upper Bound'] = Q3 + 1.5*IQR
#
outliers['Outliers Count'] = [
    ((dataset_features[col] < outliers.loc[col, 'Lower Bound']) |
     (dataset_features[col] > outliers.loc[col, 'Upper Bound'])).sum()
    for col in dataset_features.columns
]
return outliers

# Print the result of detect_outliers
print("\nOutlier Analysis:")
print(detect_outliers(crops_features), '\n')

```



Outlier Analysis:

Index	Lower Bound	Upper Bound	Outliers Count
Ph	-1933.000000	5799.000000	0
K	4.175000	7.415000	132
P	-130.000000	726.000000	156
N	-6.880000	16.800000	455
Zn	-0.017000	0.378200	21
S	-0.340000	3.500000	221
QV2M-W	-3.030750	24.531250	53
QV2M-Sp	6.468333	10.375000	0
QV2M-Su	6.788333	11.095000	0
QV2M-Au	9.320000	14.946667	0
T2M_MAX-W	8.225000	13.358333	0
T2M_MAX-Sp	20.846667	32.473333	187
T2M_MAX-Su	25.440000	34.586667	187
T2M_MAX-Au	20.368333	28.808333	0
T2M_MIN-W	19.055000	27.655000	0
T2M_MIN-Sp	1.600000	12.693333	0
T2M_MIN-Su	4.861667	16.688333	236
T2M_MIN-Au	7.868333	16.041667	0
PRECTOTCORR-W	4.600000	14.360000	236
PRECTOTCORR-Sp	-0.835000	3.818333	0
PRECTOTCORR-Su	0.311667	8.138333	0
PRECTOTCORR-Au	2.941667	26.741667	0
WD10M	-0.076667	9.363333	0
GWETTOP	-42.315000	166.685000	268
CLOUD_AMT	0.615000	0.815000	129
WS2M_RANGE	32.885000	74.845000	357
PS	4.165000	7.485000	281
	73.460000	83.300000	121

```
custom_palette = ['#2c5530', '#739e82', '#f3ffb6', '#d38b5d', '#99621e']
```

```
%config InlineBackend.figure_format='retina'
```

```

custom_params = {'grid.alpha': 0.5,
                  'axes.titlesize': 12,
                  #'figure.figsize': (10, 5),
                  'figure.titlesize': 14,
                  'axes.spines.top': False, # Remove the top and right spines
                  'axes.spines.right': False, # Remove the top and right spines
                  'axes.labelsize': 10,
                  'xtick.labelsize': 8,
                  'ytick.labelsize': 8,
                  }

```

```

sns.set_theme(context='paper', style='whitegrid', palette='BrBG', font='sans-serif', font_scale=1, color_codes=True,
               rc=custom_params)

```

```
# Create correlation matrix
correlation_matrix = data.drop(columns=['Soilcolor','label'], axis=1).corr()

# Visualize correlations
fig, axes = plt.subplots(figsize=(20,20), dpi=100)
sns.heatmap(correlation_matrix, annot=True, cmap=custom_palette, center=0)
plt.title('Feature Correlation Matrix')
# Adjust label sizes
plt.xticks(fontsize=8) # Smaller x-axis labels
plt.yticks(fontsize=8) # Smaller y-axis labels
plt.title('Feature Correlation Matrix', fontsize=10) # Smaller title
plt.show()
```



```
# Visualize distributions
fig, axes = plt.subplots(5, 5, figsize=(20,20))

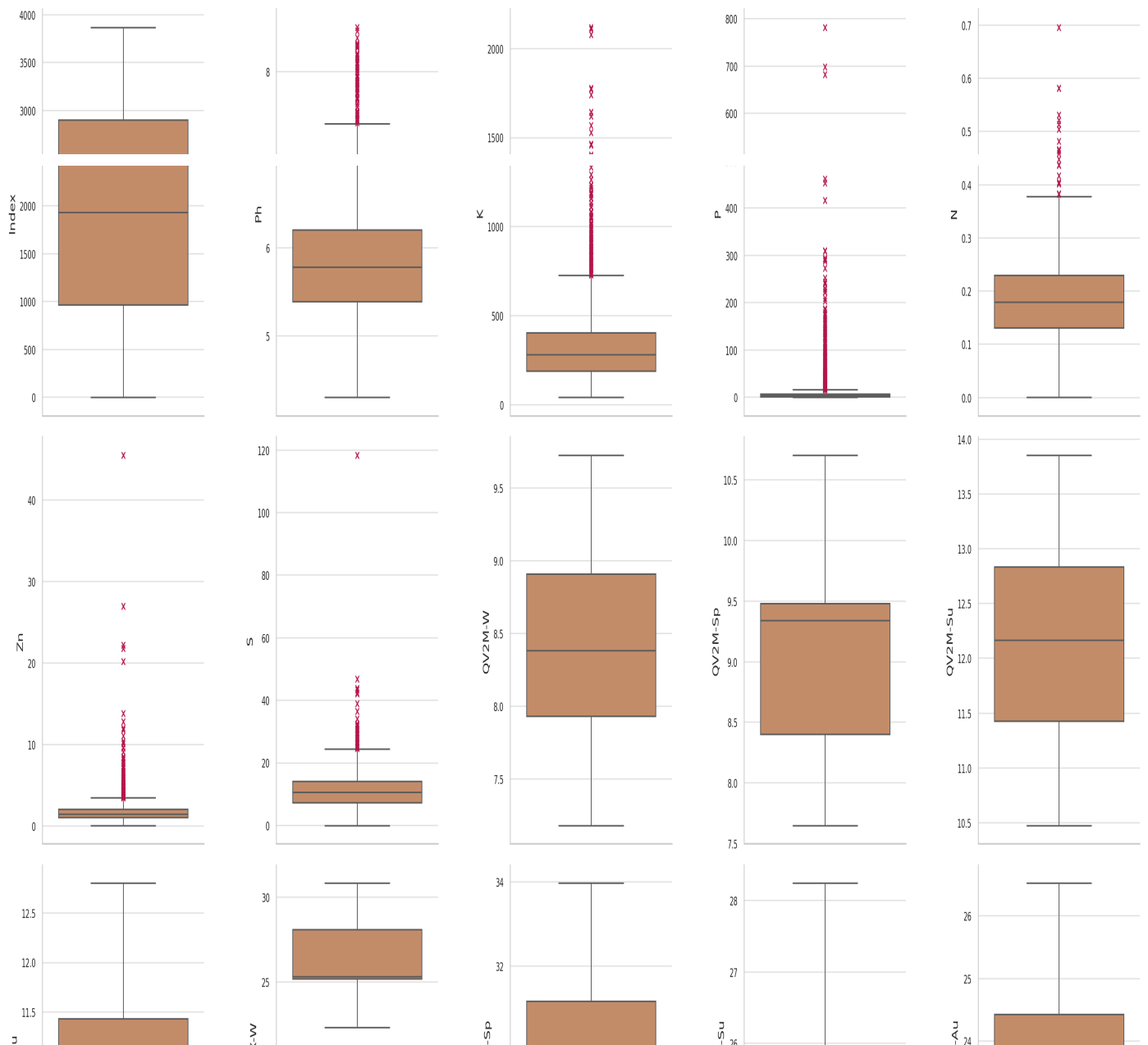
for ax, col in zip(axes.flat[:len(crops_features.columns)], crops_features.columns): # .flat converts a multi-dimensional array
    # Create a boxplot
    sns.boxplot(data=crops_features, y=col, ax=ax, color=custom_palette[3], flierprops={'markerfacecolor': '#b6174b',
        'markeredgecolor': '#b6174b',
        'marker': 'x',
        'markersize': 4})

axes.flat[-1].remove()
# Add figure title
```

```
fig.suptitle('Feature Distributions and Outlier Patterns', y=1)
plt.tight_layout(h_pad=1, w_pad=5) # Increase spacing between subplots
plt.show()
```



Feature Distributions and Outlier Patterns



```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
label_encoder_soil = LabelEncoder()
label_encoder_label = LabelEncoder()
data['Soilcolor'] = label_encoder_soil.fit_transform(data['Soilcolor'])
data['label'] = label_encoder_label.fit_transform(data['label'])
```

```
X = data.drop(columns=['label'])
y = data['label']
```

```
#value 0 or 1 ko bicha ma auna (normalize gareko)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

```
from sklearn.metrics import classification_report, accuracy_score
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
def evaluate_model(model, X_test, y_test):
    # Predicting using the provided model
    y_pred = model.predict(X_test)

    # Calculating accuracy
    score = accuracy_score(y_test, y_pred)
    print(f"Model Accuracy on Test Data = {score*100:.2f}%")

    # Printing classification report
    print(f"Classification Report {model} :\n{classification_report(y_test, y_pred)}")

    # Calculating and printing confusion matrix
    cf_matrix = confusion_matrix(y_test, y_pred)
    print(f"Confusion Matrix:\n{cf_matrix}")

    # Plotting the confusion matrix
    fig, ax = plt.subplots(figsize=(6,6))
    sns.heatmap(cf_matrix, annot=True, cmap='coolwarm')
    fig.suptitle("Confusion Matrix", color="orange", fontsize=16)
    ax.set(xlabel="Predicted Label", ylabel="Actual Label")
    plt.show()
```

```
from sklearn.ensemble import RandomForestClassifier
#creating an object for randomforestclassifier
clf = RandomForestClassifier(random_state=42)
#run
clf.fit(X_train, y_train)
y_predrandom = clf.predict(X_test)

# Evaluating the model
score = accuracy_score(y_test, y_predrandom)
print(f"Model Accuracy on Test Data = {score*100:.2f}%")
```

➦ Model Accuracy on Test Data = 93.410853%

```
evaluate_model(clf,X_test,y_test)
```

```

Model Accuracy on Test Data = 93.41%
Classification Report RandomForestClassifier(random_state=42) :

```

	precision	recall	f1-score	support
0	0.97	0.96	0.96	121
1	0.79	0.83	0.81	41
2	0.88	1.00	0.94	15
3	0.00	0.00	0.00	3
4	0.87	0.94	0.90	141
5	0.75	0.60	0.67	15
6	0.79	0.73	0.76	15
7	0.40	0.33	0.36	6
8	0.67	0.44	0.53	9
9	0.56	0.36	0.43	14
10	1.00	1.00	1.00	254
11	1.00	1.00	1.00	140
accuracy			0.93	774
macro avg	0.72	0.68	0.70	774
weighted avg	0.93	0.93	0.93	774

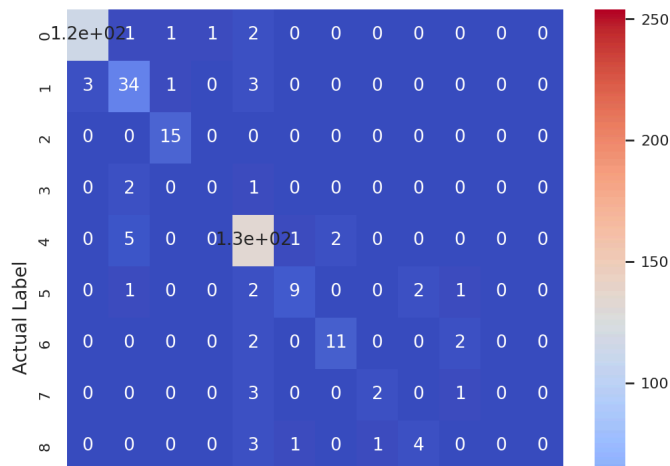
Confusion Matrix:

```

[[116  1  1  1  2  0  0  0  0  0  0  0]
 [  3 34  1  0  3  0  0  0  0  0  0  0]
 [  0  0 15  0  0  0  0  0  0  0  0  0]
 [  0  2  0  0  1  0  0  0  0  0  0  0]
 [  0  5  0  0 133  1  2  0  0  0  0  0]
 [  0  1  0  0  2  9  0  0  2  1  0  0]
 [  0  0  0  0  2  0 11  0  0  2  0  0]
 [  0  0  0  0  3  0  0  2  0  1  0  0]
 [  0  0  0  0  3  1  0  1  4  0  0  0]
 [  1  0  0  0  4  1  1  2  0  5  0  0]
 [  0  0  0  0  0  0  0  0  0  0 254  0]
 [  0  0  0  0  0  0  0  0  0  0  0 140]]

```

Confusion Matrix



```

from xgboost import XGBClassifier
xgb = XGBClassifier()
xgb.fit(X_train, y_train)
y_predxgb = xgb.predict(X_test)
score = accuracy_score(y_test, y_predxgb)
print(f"Model Accuracy on Test Data = {score*100:2f}%")

```

```

Model Accuracy on Test Data = 99.354005%

```

```

evaluate_model(xgb,X_test,y_test)

```

```

Model Accuracy on Test Data = 99.35%
Classification Report XGBClassifier(base_score=None, booster=None, callbacks=None,
    colsample_bylevel=None, colsample_bynode=None,
    colsample_bytree=None, device=None, early_stopping_rounds=None,
    enable_categorical=False, eval_metric=None, feature_types=None,
    gamma=None, grow_policy=None, importance_type=None,
    interaction_constraints=None, learning_rate=None, max_bin=None,
    max_cat_threshold=None, max_cat_to_onehot=None,
    max_delta_step=None, max_depth=None, max_leaves=None,
    min_child_weight=None, missing=nan, monotone_constraints=None,
    multi_strategy=None, n_estimators=None, n_jobs=None,
    num_parallel_tree=None, objective='multi:softprob', ...) :
precision    recall  f1-score   support

```

0	1.00	1.00	1.00	121
1	1.00	1.00	1.00	41
2	0.88	1.00	0.94	15
3	1.00	0.33	0.50	3
4	1.00	1.00	1.00	141
5	1.00	1.00	1.00	15
6	1.00	0.93	0.97	15
7	0.86	1.00	0.92	6
8	1.00	0.89	0.94	9
9	0.93	1.00	0.97	14
10	1.00	1.00	1.00	254
11	0.99	1.00	1.00	140

accuracy			0.99	774
macro avg	0.97	0.93	0.94	774
weighted avg	0.99	0.99	0.99	774

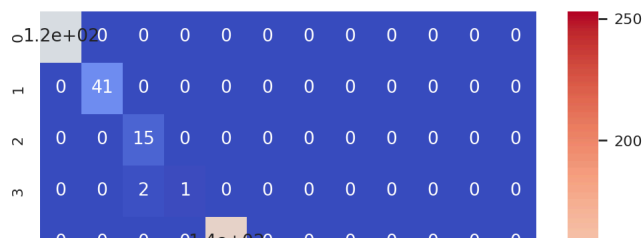
Confusion Matrix:

```

[[121  0  0  0  0  0  0  0  0  0  0  0]
 [  0 41  0  0  0  0  0  0  0  0  0  0]
 [  0  0 15  0  0  0  0  0  0  0  0  0]
 [  0  0  2  1  0  0  0  0  0  0  0  0]
 [  0  0  0  0 141  0  0  0  0  0  0  0]
 [  0  0  0  0  0 15  0  0  0  0  0  0]
 [  0  0  0  0  0  0 14  1  0  0  0  0]
 [  0  0  0  0  0  0  0  6  0  0  0  0]
 [  0  0  0  0  0  0  0  0  8  1  0  0]
 [  0  0  0  0  0  0  0  0  0 14  0  0]
 [  0  0  0  0  0  0  0  0  0  0 253  1]
 [  0  0  0  0  0  0  0  0  0  0  0 140]]

```

Confusion Matrix



```

from sklearn.neighbors import KNeighborsClassifier
KNN = KNeighborsClassifier()
KNN.fit(X_train, y_train)
y_predKNN = KNN.predict(X_test)
score = accuracy_score(y_test, y_predKNN)
print(f"Model Accuracy on Test Data = {score*100:2f}%")

```

Model Accuracy on Test Data = 77.519380%

```
evaluate_model(KNN,X_test,y_test)
```

Model Accuracy on Test Data = 77.52%

Classification Report KNeighborsClassifier() :

	precision	recall	f1-score	support
0	0.79	0.78	0.78	121
1	0.37	0.37	0.37	41
2	0.67	0.40	0.50	15
3	0.00	0.00	0.00	3
4	0.69	0.84	0.76	141
5	1.00	0.07	0.12	15
6	0.56	0.33	0.42	15
7	0.00	0.00	0.00	6
8	0.00	0.00	0.00	9
9	0.14	0.07	0.10	14
10	0.81	0.06	0.09	251